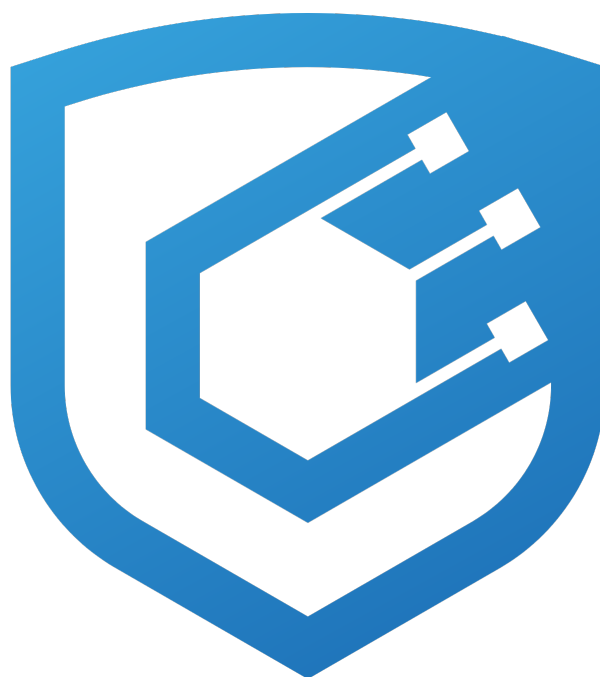




Atum V2 EVM/TRON Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Dacian](#)

[Jorge](#)

October 17, 2025

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
6.1	Centralization and Trust Assumptions	3
6.1.1	Token Issuer Risks	3
6.1.2	Protocol Admin Powers	3
6.1.3	Security Boundaries	3
6.1.4	Protocol Trust Model	4
6.2	Formal Verification	4
6.2.1	Deposit Parameter Tampering Protection	4
6.2.2	Additional Verified Properties	5
6.3	Interesting Observations	5
7	Findings	7
7.1	Informational	7
7.1.1	Batch functions should revert on empty arrays	7
7.1.2	Don't initialize to default values in Solidity	7
7.1.3	Events emitted using <code>indexed</code> structs can be harder to query than using simple types	7

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Atum is a cross-chain stablecoin payments protocol that leverages an off-chain Request for Quote (RFQ) mechanism to facilitate seamless token transfers across different blockchains and stablecoin types. The protocol's innovative design abstracts away blockchain complexity from the end users (Depositors), who simply need to sign a Permit2 message to initiate a payment request - they never need to interact directly with smart contracts or manage cross-chain messaging.

When a Depositor needs to pay in stablecoins across chains (e.g., deposit USDC on Ethereum but pay using DAI on Arbitrum), they publish an RFQ containing their signed Permit2 authorization, which multiple liquidity providers (Settlers) can bid on competitively. The winning Settler, designated by a trusted Reserver, locks the source funds in the Escrow contract and fulfills the swap on the destination chain, after which a trusted Releaser verifies the fulfillment and authorizes the release of escrowed funds to the Settler.

This architecture pushes all technical complexity - including liquidity management, cross-chain execution, and blockchain interactions - to sophisticated Settlers who compete to provide the best rates, while Depositors enjoy a simple, signature-based experience. The protocol employs a trust-minimized design where the Escrow contract ensures atomic settlement through cryptographic signatures and role separation, with the Reserver controlling Settler assignment and the Releaser validating cross-chain fulfillment, making it particularly suitable for applications like cross-border payments, remittances, and chain-agnostic stablecoin transfers where user simplicity is paramount.

5 Audit Scope

Atum V1 for EVM, TRON & Solana was previously audited by Cyfrin; the purpose of this audit is to evaluate the following V2 EVM/TRON changes:

- ```
1) New function call flow: deposit -> refund|release
 - reserve function no longer exists, instead the reserve is done in the original deposit
 - both the deposit sig and the reserve sig are passed into the deposit function
```

- 2) Storage compression. Included in this:
  - new witness strings & hashes
  - more parameters in each witness for reserve, release and refund - to reconstruct the hash and check  
↳ against storage value during execution
- 3) To handle edge case with Gnosis Safe smart contract wallets.depositId is derived using:
  - keccak256(abi.encode(depositor, signature, permit.nonce));, instead of:
  - keccak256(abi.encode(depositor, signature));
- 4) Deadline parameters removed from Release and Refund witnesses
- 5) New functions to batch each operation: depositMany, releaseMany, refundMany
- 6) Event structure changes to all events (Deposit, DepositedMany, Released, ReleasedMany, Refunded, RefundedMany, Fulfilled, FulfilledMany)  
↳ RefundedMany, Fulfilled, FulfilledMany
- 7) New fulfillMany function

The audit scope was limited to:

```
1-atum-evm/src/Escrow.sol
1-atum-evm/src/FulfillmentProxy.sol
2-atum-tvm/contracts/Escrow.sol
2-atum-tvm/contracts/FulfillmentProxy.sol
2-atum-tvm/contracts/TIP712.sol
```

## 6 Executive Summary

Over the course of 5 days, the Cyfrin team conducted an audit on the [Atum V2 EVM/TRON](#) smart contracts provided by [Atum Labs](#). In this period, a total of 3 issues were found.

The only findings were 3 informational issues.

### 6.1 Centralization and Trust Assumptions

The protocol is designed to work with centralized stablecoins that implement blacklist functionality. This introduces several trust assumptions and centralization vectors:

#### 6.1.1 Token Issuer Risks

Both protocol contract addresses and participant addresses (Depositor, Reserver, Releaser, Settler) can be blacklisted by token issuers, potentially freezing tokens indefinitely within the protocol. This creates an external dependency where the protocol's functionality can be disrupted by entities outside the protocol's control.

#### 6.1.2 Protocol Admin Powers

Protocol admins (contract owner) have limited but important powers:

- **Token Management:** Enable and disable supported tokens via the allowlist. Note that disabling a token does not affect existing active deposits, allowing them to complete or be refunded normally
- **Emergency Controls:** Pause and unpaue the `Escrow` contract, halting all user operations

#### 6.1.3 Security Boundaries

Importantly, protocol admins **cannot**:

- Withdraw or "rescue" tokens from the `Escrow` contract (no backdoor functions exist)

- Upgrade the contract implementation (contract is immutable)
- Alter or interfere with active deposits
- Override signature requirements or bypass the cryptographic validation

#### 6.1.4 Protocol Trust Model

The protocol involves the following on-chain actors:

- **Depositor** - users requesting cross-chain payments
- **Reserver** - entity which chooses **Settler** on behalf of **Depositor**
- **Releaser** - trusted entity who can release or refund escrowed funds
- **Settler** - address which receives escrowed funds if **Releaser** chooses to release them

**Depositor** chooses **Reserver** and must trust **Reserver** to choose the **Settler** which offers the best deal for the requested service. Beyond this however there is no further need for **Depositor** and **Reserver** to trust each-other as:

- both parties must agree on important parameters for every deposit including token address, amount and who **Releaser** is
- both parties cryptographically sign important parameters using EIP-712 signatures
- both EIP-712 signatures are validated on-chain before deposit creation
- **Depositor** funds are always deposited into **Escrow**; **Reserver** can't redirect funds elsewhere
- the only power **Reserver** has is to designate **Settler**

Hence the **Depositor** <-> **Reserver** relationship is cryptographically secured and almost entirely trustless.

However both **Depositor** and **Reserver** must completely trust **Releaser** to behave correctly because once the funds are deposited into **Escrow** the **Releaser** is free to:

- **Never Refund** - depositor funds stuck in **Escrow** even though **Settler** never fulfilled the cross-chain payment request
- **Never Release** - **Settler** never receives payment for fulfilling the cross-chain payment request
- **Refund After Fulfillment** - **Depositor** gets their escrowed tokens back even though **Settler** fulfilled the cross-chain payment request
- **Release Without Fulfillment** - **Settler** receives payment even though they didn't fulfill the cross-chain payment request

Initially the protocol will operate all **Releaser** entities such that this trust assumption is valid. Additionally **Releaser** can't seize escrowed funds or change addresses where tokens are sent; **Releaser** can only do nothing, refund to **Depositor** or release to **Settler**.

## 6.2 Formal Verification

Using Certora we have formally verified a number of useful properties and provided the configuration and specification files to the client as an additional deliverable.

### 6.2.1 Deposit Parameter Tampering Protection

The most significant change in Atum V2 is that:

- deposit ids are stored on-chain against a hash of the deposit parameters
- **release** and **refund** operations take the deposit parameters as input, reconstruct the expected hash and verify it matches what is stored on-chain

This is a more gas-efficient implementation which protects against tampering with the deposit parameters so that `Releaser` is unable to redirect funds to an arbitrary address, change token or amount, or otherwise modify any deposit parameters.

### 6.2.2 Additional Verified Properties

The following additional important properties were also formally verified:

- **Deposit Existence** - `release` and `refund` can't operate on non-existent deposits
- **One-Time Execution & Replay Protection** - successful `release` or `refund` clears storage such that subsequent `release` or `refund` will revert for the successfully processed deposit id
- **Exact Token Transfer** - `release` and `refund` transfer the exact amount of tokens to settler and depositor respectively

## 6.3 Interesting Observations

When reconstructing the active deposit hash, the set of parameters used (`depositor`, `token`, `amount`, `re-server`, `releaser`, `settler`) could theoretically be identical for multiple different active deposits; a state could exist where multiple unique deposit ids map to the same active deposit hash.

Fulfillment on the destination chain uses the unique deposit id as `originReference` so the off-chain matching of fulfillments to deposits should be relatively simple. As mentioned in the Protocol Trust Model the `Releaser` is trusted to act correctly and `release` or `refund` deposits correctly.

Hence there doesn't appear to be any negative consequences to a state where multiple active unique deposit ids map to the same deposit hash.

### Summary

|                |                                    |
|----------------|------------------------------------|
| Project Name   | Atum V2 EVM/TRON                   |
| Repository     | <a href="#">evm-contracts</a>      |
| Commit         | <a href="#">ced6f29bea29...</a>    |
| Fix Commit     | <a href="#">71206096773b...</a>    |
| Repository 2   | <a href="#">tvm-contracts</a>      |
| Commit         | <a href="#">a32d60551c65...</a>    |
| Fix Commit     | <a href="#">ee83857255f9...</a>    |
| Audit Timeline | Oct 13th - Oct 17th, 2025          |
| Methods        | Manual Review, Formal Verification |

### Issues Found

|                   |   |
|-------------------|---|
| Critical Risk     | 0 |
| High Risk         | 0 |
| Medium Risk       | 0 |
| Low Risk          | 0 |
| Informational     | 3 |
| Gas Optimizations | 0 |
| Total Issues      | 3 |

### Summary of Findings

|                                                                                                        |              |
|--------------------------------------------------------------------------------------------------------|--------------|
| [I-1] Batch functions should revert on empty arrays                                                    | Acknowledged |
| [I-2] Don't initialize to default values in Solidity                                                   | Resolved     |
| [I-3] Events emitted using <code>indexed</code> structs can be harder to query than using simple types | Acknowledged |

## 7 Findings

### 7.1 Informational

#### 7.1.1 Batch functions should revert on empty arrays

**Description:** The newly added batch functions `Escrow::depositMany`, `releaseMany`, `refundMany` and `FulfillmentProxy::fulfillMany` should revert when their input arrays are empty; the current implementations will emit empty non-sensical events.

**Atum:** Acknowledged; prefer lower gas costs by not including empty array checks.

#### 7.1.2 Don't initialize to default values in Solidity

**Description:** Don't initialize to default values in Solidity:

```
FulfillmentProxy.sol
23: for (uint256 i = 0; i < params.length; i++) {
```

**Atum:** Fixed in commit [f899054](#) for EVM and [ee83857](#) for TRON.

**Cyfrin:** Verified.

#### 7.1.3 Events emitted using indexed structs can be harder to query than using simple types

**Description:** In Atum V1 events were emitted using indexed simple types eg:

```
event Deposited(
 bytes32 indexed requestId,
 bytes32 indexed depositId,
 address indexed depositor,
 address reserver,
 address releaser,
 IERC20 token,
 uint256 amount
);
```

This made is very easy to query for all deposits belonging to a given `depositor` address.

But in Atum V2 events are now emitted using indexed structs:

```
event Deposited(
 bytes32 indexed depositId, DepositWitness indexed depositWitness, ReserveWitness indexed
 ↳ reserveWitness
);
```

Now it is much harder to query for all deposits belonging to a given `depositor` address, since when a struct is marked as indexed, it is treated as a complex type. The topic (used for filtering) stores the Keccak-256 hash of a special in-place encoding of the entire struct (concatenation of its members' encodings, padded to multiples of 32 bytes).

To query, you must know the exact values of all fields in the struct to compute the matching hash and filter by it. Partial matches (e.g., filtering by just one field) are impossible using the topic alone—you'd need to fetch broader logs and filter off-chain, which is less efficient.

Consider "unpacking" structs when emitting events and indexing the most important individual basic fields used in queries.

**Atum:** Acknowledged.